Python Data Storage

DATA 118 Lab — Student Version

Goals

The goal of this lab is review some of what you've learned about coding in Python. We will cover:

- Lists
- Sets
- Tuples
- Arrays
- Dictionaries

Storing Data in Python

Python and its libraries provide many ways to group data together. Some important ones:

- Lists, Tuples, Sets, Dictionaries (built-in to Python)
- Arrays (found in the `numpy` library)
- DataFrames (in `pandas`)

The above are listed in order of increasing functionality and sophistication. In general you should use the simplest one that meets your needs.

Lists

A **list** is an ordered sequence of values that can be changed.

You can create a list by using square brackets "`[]`". Each element is separated by commas.

Let's create a list!

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]
School_locations
```

Lists can have different data types!

Exercise

```
Random_stuff = ["California", 38, 3.14159] #A string, int, float
Random_stuff
```

Built-in Functions Recall Python has built-in functions of the form `function_name()` A very common one is the `print()` function!

`type()` is another important function. We can call this on a number, string, list, or any object to see type it is!

Exercise

```
print("California")
print(type("California"))
print(type(38))
print(type("38"))
print(type(3.14))
print(type(Random_stuff))
```

We can also find the length of a list:

Exercise

```
len(Random_stuff)
```

Accessing values from a list Each element of a sequence is assigned an index corresponding to its position where **indices start at 0**. We can access an element by calling the sequence or list and putting in square brackets the element we want!

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
↪  "Texas", "Georgia", "Washington D.C."]
School_locations[1]
```

Negative indexes count from the end of the sequence.

Exercise

```
School_locations[-1]
```

If our list is long or we want the first or second from end, this is useful!

1. What will be the result of running the following code? Answer in the box below.

```
School_locations = ["California", "Illinois", "North Carolina", "Texas",
 ↪  "Georgia", "Washington D.C."]

School_locations[6]
```

Question 1. What will be the result of running the code above?
☐ 'California'
☐ 'Illinois'
☐ 'North Carolina'
☐ 'Texas'
☐ 'Georgia'
☐ 'Washinton D.C.'
☐ An error

List Slicing We can also extract a "slice" of a list.

The range of elements can be specified with colons. The output is a list starting at the left index and stopping at (right index -1). The range you specify is a half-closed interval: [start,end).

```
list[start: end]
```

Exercise

```
School_locations[1:3] # up to but not including the end of the slice
```

The above prints out the second and third elements.

We can still slice a list by 'leaving out' a starting (or ending) position. The missing position will revert to the default start value: the beginning of the list. The default end value is the end of the list.

Exercise

```
School_locations[:3] #start at beginning stop at index end-1 -----> 3-1=2
```

2. What will the following output? Answer in the box below.

```
School_locations = ["California", "Illinois", "North Carolina", "Texas",
 ↪  "Georgia", "Washington D.C."]

School_locations[:]
```

Question 2. What will be the result of running the code above?
☐ `[]`
☐ `['California', 'Washington D.C.']`
☐ `['California', 'Illinois', 'North Carolina', 'Texas', 'Georgia', 'Washington D.C.']`
☐ An error

Default Settings: List Slicing It might be useful to take out every even index from a list.

There is an optional argument we can use when slicing a list, the step. In general we can slice by using `list[start: end: step]` where the default (what is used when it isn't specified) of step is 1

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]

School_locations[::2]
```

3. What do you think the following will output? Answer in the box below.

```
School_locations[:3:3]
```

Question 3. What will be the result of running the code above?
☐ `[]`
☐ `['California']`
☐ `['California', 'Texas']`
☐ `['North Carolina', 'Washington D.C.']`
☐ `['North Carolina']`
☐ `['California', 'Illinois', 'North Carolina', 'Texas', 'Georgia', 'Washington D.C.']`
☐ An error

Operations and manipulation on lists We can:

- Insert items into lists!
  - At either end
  - In the middle

- Count how many items have a specific value.
- Sort elements.

How do we do this? By using *methods*. *Methods* are particular built-in functions that work on objects in Python. There are specific methods that work for all *list* objects!

List methods take the form:

```
list.method()
```

Built-in functions can also be applied to objects (including lists) in Python. Recall that a function applied to a list will take the form:

```
function_name(list)
```

Appending to a list You can append an item to the end of a list:

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]

School_locations.append('Michigan')
School_locations
```

Or you can insert a value at a particular index:

Exercise

```
School_locations.insert(4,'tomato')
School_locations
```

Note we didn't assign the result to `School_locations`, but this **automatically changes/overwrites** the object.

Let's make a copy of our list so we can see how this works.

Exercise

```
School_location_og = School_locations.copy()
School_location_og
```

4. Append the number 2023 to `School_locations` and then view the list. Note even though the list contains only strings, I can append an integer as lists accept any data type.

Exercise

```
...

School_locations
```

This does not change our copy of the list!

Exercise

```
School_location_og
```

How do we find methods? Use online documentation!

Or use the built-in function `dir()`.

Exercise

```
print(dir(School_locations))

print(dir(list))
```

Let's play with lists! Create a list with the following words as strings: classify, evaluate, great, dorm, inside. Name the list `my_list`.

Exercise

```
my_list
```

Remove the 2nd and 3rd elements. You may want to check the documentation for list functions and methods! Because of how we are parsing your solutions, please leave the `my_list` line as is!

Exercise

```
...

my_list
```

Append 316 to the end of the list.

Exercise — write your code here:

Insert the string "Math is fun" at index 2.

Exercise — write your code here:

Question 4. Did your above operations change your original list, or create a copy?
☐ Change Original
☐ Create a Copy

Given list `number_list = [1,2,3,4,5,6,7]`, create a new list `new_list = [6,4,2]` in 2 lines of code.

Exercise

```
number_list = [1,2,3,4,5,6,7]
```

Sets

Let's say that you want to find all unique elements in the following list:

Exercise

```
grocery_list = ['apple', 'apple', 'milk', 'cheese', 'egg', 'egg',
 ↪  'egg']
```

The best way to do this with a list is to write a loop or list comprehension. However, there is a better way to carry out tasks like this using a different data type: a **set**. A **set** is an unordered sequence of unique values that can be changed.

You can create a set by using the curly braces ({}). Each element is separated by commas. Alternatively, you can use the built-in function `set()` to convert a list to a set.

Let's create a set! First, let's try using the curly braces:

Exercise

```
grocery_set = {'apple', 'apple', 'milk', 'cheese', 'egg', 'egg', 'egg'}
grocery_set
```

Now, let's see what using `set()` looks like:

Exercise

```
grocery_set = set(grocery_list)
grocery_set
```

Count how many times each item appears in `grocery_list`, and then how many times each item appears in `grocery_set`. Notice that the duplicate values are removed, and you are left with only unique values. This is one of the advantages of sets!

Properties of Sets Sets borrow mathematical concepts as a way to maintain information on different things and possibly counts.

`in` One of the most helpful operators you can use on sets are logical operators, like `in`, which will tell you if an item is included in the set:

Exercise

```
'cheese' in grocery_set
'bread' in grocery_set
```

Intersect You can also find all of the elements that two sets have in common. This is known as finding the intersection of two sets, and uses the &:

Exercise

```
fruits_set = {'apple', 'grape', 'kiwi', 'pear', 'strawberry'}
fruits_set & grocery_set
```

Question 5. If we create a third set, `dairy_set = {'milk', 'cheese', 'cream'}`, what will the intersection of `groceries_set` and `dairy_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Union You can also find all of the elements that two sets have in total, even if the item only belongs to one set. This is known as finding the union of two sets, and uses the |:

Exercise

```
fruits_set | grocery_set
```

Question 6. Again, let `dairy_set = {'milk', 'cheese', 'cream'}`. What will the union of `groceries_set` and `dairy_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Difference Finally, you can find all of the elements in one set, but not the other. This is known as finding the difference between two sets, and uses the `-`:

Exercise

```
fruits_set - grocery_set
grocery_set - fruits_set
```

Question 7. Again, let `dairy_set = {'milk', 'cheese', 'cream'}`. What will `dairy_set - grocery_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Sets can have different data types! Just like lists!

Exercise

```
Random_set = {"California", 38, 3.14159} #A string, int, float
Random_set
```

`type()` Remember that we can call `type()` on an object, including sets, to see type it is!

Exercise

```
print(type(Random_set))
```

We can also find the size of a set:

Exercise

```
len(Random_set)
```

Accessing values from a set Sets are *unordered*. Therefore, elements of a set are not assigned any indices. This means that you cannot access an element in a set by indexing!

Operations and manipulation on sets We can:

- Insert items into sets!
- Remove items from sets!

How do we do this? By using the built-in set methods.

Adding to a set You can add an item to a set with the `.add()` method:

Exercise

```
grocery_set.add('bread')
grocery_set
```

Note we didn't assign the results to `grocery_set`. Just like lists, the output was automatically changed.

Your turn! Try adding `'butter'` to `grocery_set`.

Exercise — write your code here:

Removing from a set You can remove an item from a set with the `.discard()` method:

Exercise

```
grocery_set.discard('egg')
grocery_set
```

Subsets A set where all of the elements belong to a larger set is known as a subset. You can check to see if one set is a subset of another using the method `.issubset()`.

Exercise

```
toast_set = {'bread', 'butter'}
toast_set.issubset(grocery_set)

toast_set.add('cinnamon')
toast_set.add('sugar')
toast_set.issubset(grocery_set)
```

How do we find methods? Use the built-in function `dir()`.

Exercise

```
print(dir(grocery_set))
```

Let's play with sets! Use the following two sets for the exercises below:

```
my_set_1 = {'apple', 'banana', 'cherry', 'date'}
my_set_2 = {'elderberry', 'fig', 'grape', 'honeydew'}
```

1. Confirm that these sets have no elements in common, i.e., are *disjoint*. You may want to look up a new method!

Exercise — write your code here:

2. Create a new set that contains all of the elements in `my_set_1` and `my_set_2`. Call this set `my_big_set`.

Exercise — write your code here:

3. Confirm that `my_set_1` is a subset of `my_big_set`.

Exercise — write your code here:

4. Write a line of code to see if `'plum'` is in `my_big_set`.

Exercise — write your code here:

Tuples

Like lists, a **tuple** is an ordered sequence of values. However, unlike lists or sets, tuples cannot be changed (including adding and removing elements) after they are created–this is known as being **immutable**.

Tuples can be helpful when data are structured like Cartesian coordinates, but can also be generalized to a greater number of dimensions. These objects often can be stored together, but are somewhat nonsensical to store as a list.

Consider a list of universities with the following information:

- Name
- City
- State

It might make more sense to bundle this information as a tuple (also called "packing").

You can create a tuple by using parentheses (`()`). Each element in the tuple should be separated by commas.

Let's create a tuple! Let's create a tuple representing the University of Chicago and its city and state.

Exercise

```
uc = ("University of Chicago", "Chicago", "Illinois")
uc
```

Tuples can have different data types! Let's say that we want to add an element to the tuple for the University's undergraduate enrollment (which requires an integer type, not a string). Remember, you can't change a tuple after it has been made because it is immutable! This includes adding or removing elements. You would have to overwrite the tuple or create a new one.

Exercise

```
uc_enrollment = ("University of Chicago", "Chicago", "Illinois", 7653)
uc_enrollment
```

`type()` Remember that we can call `type()` on an object, including sets, to see what type it is!

Exercise

```
type(uc_enrollment)
```

We can also find the length of a tuple:

Exercise

```
len(uc_enrollment)
```

Accessing values from a tuple Because tuples are ordered, each element of the tuple is assigned an index corresponding to its position where **indices start at 0**. We can access an element by calling the tuple and putting in square brackets the element we want!

Exercise

```
uc_enrollment[1]
```

Just like lists, negative indexes count from the end of the tuple.

Exercise

```
uc_enrollment[-1]
```

9. What will be the result of running the following code? Answer in the box below.

```
stanford_enrollment = ("Stanford University", "Stanford", "California",
↪  7554)
stanford_enrollment[2]
```

Question 8. What will be the result of running the code above?
☐ `'Stanford University'`
☐ `'Stanford'`
☐ `'California'`
☐ `7554`

Tuple methods Python only has two built-in methods for tuples!

- `count()`: Returns the number of times a specified value occurs in a tuple.
- `index()`: Searches the tuple for a specified value and returns the position of where it was found

Write a line of code to count how many times `'Illinois'` occurs in `uc_enrollment`.

Exercise — write your code here:

Write a line of code to find which index represents `'Illinois'` in `uc_enrollment`.

Exercise — write your code here:

Nested sequences Create a new tuple, `northwestern_enrollment`, in the same form as `uc_enrollment`, knowing that Northwestern University is located in Evanston, Illinois, and has an undergraduate enrollment of 8,709.

Exercise — write your code here:

It would be useful to store our two college-related tuples together along with any we create in the future. Create a list containing these two tuples in alphabetical order by college name. Let's call the list `tuple_list`.

Exercise

```
tuple_list = ...
tuple_list
```

Dictionaries

A **dictionary** is a set of "key: value" pairs where each key is unique. We can create a dictionary with curly brackets "{}". Entries of a dictionary are of the form "key: value".

Exercise

```
survey_dict = {0: "Strongly Disagree", 1: "Disagree", 2: "No opinion",
 ↪  3: "Agree", 4: "Strongly Agree"}
```

We can extract values of a dictionary by their key–this is why all keys must be unique!

Exercise

```
survey_dict[1]
```

Dictionaries are useful for storing and extracting data! Think of them like an address book where you look up someone's address by finding their name.

A few useful operations with dictionaries:

- Add an entry
- Delete an entry

We can delete pairs:

Exercise

```
del survey_dict[1]
survey_dict
```

Or add new pairs:

Exercise

```
survey_dict['new_key'] = 'new value'
survey_dict
```

We can also determine if keys are contained in the dictionary

Exercise

```
3 in survey_dict
```

Exercise

```
"Disagree" in survey_dict
```

Exercise

```
'new_key' in survey_dict
```

Keep in mind, keys do not all need to be the same type...although it may make more sense to keep them that way.

Exercise

```
survey_dict_2 = {"Strongly Disagree": 0 , "Disagree": 1 , 2: "No
↪  opinion", 3: "Agree", 4: "Strongly Agree"}
```

Exercise

```
survey_dict_2["Disagree"]
```

Exercise

```
list(survey_dict_2)
```

Dictionary Methods

There are methods that you can use on dictionaries, just like with lists and arrays. Here are a few:

Exercise

```
print(dir(survey_dict_2))
```

Exercise

```
survey_dict_2.keys()
```

Exercise

```
survey_dict_2.items()
```

Working with Dictionaries

Below is a dictionary containing total number of homicides in the United States in 2021, by state. (Published by Statistal Research Department, Oct 14, 2022). Note Washington D.C. is included as 'District of Columbia'.

Exercise

```
homicide_dict = {'Texas': 2064, 'North Carolina': 928, 'Ohio': 824,
 ↪  'Michigan': 747, 'Georgia': 728, 'Tennessee': 672, 'Missouri':
 ↪  593, 'Virginia': 562, 'South Carolina': 548, 'Illinois': 514,
 ↪  'Pennsylvania': 510, 'Louisiana': 447, 'Indiana': 438, 'Alabama':
 ↪  370, 'Kentucky': 365, 'Colorado': 358, 'Washington': 325,
 ↪  'Arkansas': 321, 'Wisconsin': 315, 'Oklahoma': 284, 'Nevada': 232,
 ↪  'Minnesota': 203, 'Arizona': 190, 'Oregon': 188, 'New Mexico':
 ↪  169, 'Mississippi': 149, 'Connecticut': 148, 'Maryland': 138, 'New
 ↪  Jersey': 137, 'Massachusetts': 132, 'New York': 124, 'California':
 ↪  123, 'District of Columbia': 109, 'West Virginia': 95, 'Delaware':
 ↪  94, 'Kansas': 87, 'Utah': 85, 'Iowa': 70, 'Rhode Island': 38,
 ↪  'Idaho': 36, 'Montana': 31, 'South Dakota': 26, 'Nebraska': 25,
 ↪  'Alaska': 18, 'Maine': 18, 'Wyoming': 17, 'New Hampshire': 14,
 ↪  'North Dakota': 14, 'Vermont': 8, 'Hawaii': 6}
```

Pick a few states of interest and find their homicide number (use code here...do not just manually search the dictionary!)

Exercise — write your code here:

20. Find the number of keys in the dictionary.

Exercise — write your code here:

Discuss with a neighbor (or on Ed):

1. Does this imply all 50 states are included here?
2. What are some limitations to this data?

Arrays

Arrays (from the `NumPy` library) are another way to collect data. Like a list, they contain a sequence of values. But, unlike a list, all elements of an array must have the same data type. This is because `NumPy` arrays were built for efficient computation. They can perform operations on all elements in one step. They can also do element-wise computation. This means that, for example, if you add two arrays together, the result will be an array where the element at each index is the sum of the elements at the index in the two original arrays. For this reason, when two arrays are added (or subtracted, multiplied, or divided) they must have the same size.

Lists vs Arrays

Lists are more flexible - Can contain elements of different types

`NumPy` arrays have some advantages

- Size - they take up less computer memory than lists.
- Performance - faster access than lists.
- Functionality - linear algebra functions built-in.
- Can be multiple dimensions.

Creating an array

First import the `NumPy` library!! We do this using an import statement and we give `NumPy` the alias `np`. This makes it so that any time we want to access the `NumPy` library, we only need to type `np`.

We can create an array using `np.array()` which takes in a list of values of the same data type. Below is an example of writing the import statement and creating an empty array. The array is empty because the list has no elements.

```
import numpy as np

np.array([])
```

Now, let's actually import numpy and create an array with two elements.

Exercise

```
import numpy as np

np.array([1,2])
```

Below, we create a list that contains integers and a string.

Exercise

```
tomato_list = [22, 38, 26, 35, 35, 'tomato']
print(tomato_list)
```

Let's see what happens when we try to make an array from this list.

Exercise

```
tomato_array = np.array([22, 38, 26, 35, 35,'tomato'])
print(tomato_array)
```

A list can have different types, but an array will default to one. So you can see all `int`s were changed to `string`s (notice the single quotes around the numbers).

An `array` will make sure everything is the same type.

Exploring arrays

Let's create an array by converting a list to a `NumPy` array.

Exercise

```
prime50_array = np.array([2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
 ↪  41, 43, 47])

print(type(prime50_array))
```

Extraction and slicing of one-dimensional arrays work exactly the same as lists!

10. Print the second element of `prime50_array`.

Exercise — write your code here:

11. Print the `array` starting at index 1 and up to (but excluding) index 2.

Exercise — write your code here:

12. Now consider:

```
prime50_array[1::2]

prime50_array[::2]
```

Question 9. Will the code above give the same or different outputs?
☐ Same
☐ Different

Arrays have attributes

Attributes are characteristics of an object. We can view an object's attributes by using the dot operator `.` similarly to when we used methods by using the syntax `object.method()`. However, to access an attribute, we don't use parentheses. (See https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html)

Two attributes of arrays are size, which we can access using `.size`, and shape, accessed using `.shape`.

Exercise

```
prime50_array.size
```

Exercise

```
prime50_array.shape
```

Arrays have useful methods

Like lists, there are a lot of useful methods we can use on array objects. But unlike lists, we can always sum the elements in an array, since they are all numbers.

See the numpy documentation for a complete list of methods.

Exercise

```
print(prime50_array.sum())
```

Numpy also has many useful built-in functions to use on arrays!

Notice there is often more than one way to do a common operation!

Exercise

```
print(prime50_array.sum()) #sum method

print(np.sum(prime50_array)) #sum function (from NumPy library)

print(np.count_nonzero(np.array([1,2,0,2,1,0,2])))

print(prime50_array.mean()) #mean method

print(np.mean(prime50_array)) #mean function
```

We can easily create arrays by specifying a range. Calling `np.arange()` creates a half-closed interval [start,end) - the end value is not included

Exercise

```
np.arange(4,10)
```

If you leave out the start, the default is zero.

Exercise

```
print(np.arange(10))
```

We can specify a step size we want to increment by. If we leave out the step, the default is one

Exercise

```
print(np.arange(1,31,2))
```

Another reason why arrays are useful!

Element-wise operations!

Take a moment to think about what the following will produce before running the cell.

Exercise

```
array_1 = np.arange(10)
array_2 = np.array([1,2,3,4,5,6,7,8,9,10])
difference_array = array_1 - array_2
difference_array
```

13. Consider the following code:

```
import numpy as np
a = np.array([1,2,3,5,8])
b = np.array([0,3,4,2,1])
c = a + b
c = c*a
print(c[2])
```

Question 10. What will be the result of the code above?
Answer:

_____

14. Consider the following code:

```
number_array = np.array([1,2,3,5,8])
number_array = number_array + 1
print(number_array[1])
```

Question 11. What will be the result of the code above?
Answer:

_____

We may have higher dimensional arrays.

These are organized into rows and columns and are built row-by-row.

Exercise

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) #specify first row then
 ↪  second

print(arr_2d)
```

Exercise

```
arr_2d.shape #outputs (#rows, #columns)
```

We can reshape our arrays using `np.reshape()` which takes in the array to reshape as well as the new shape.

When the new shape provided is an integer, the array produced will be 1-dimensional.

Exercise

```
reshaped_arr = np.reshape(arr_2d, 6)

reshaped_arr.shape
```

When the new shape is a tuple, the array produced will have the same number of dimensions as elements in the tuple.

Exercise

```
np.reshape(arr_2d, (3,2))

np.reshape(arr_2d, (3,2)).shape
```

Sometimes arrays are large and we don't want to do the math to figure out the size of each dimension. If we know we want it to have 3 rows. We can use -1 for the second dimension. Using -1, tells Python to infer the remaining dimension from the size of the input array.

Exercise

```
np.reshape(arr_2d, (3,-1)) # the unspecified value is inferred to be 2

np.reshape(arr_2d, (3,-1)).shape
```

Exercise

```
arr_2d[:,2] #all of the rows, column at index 2
```

Exercise

```
arr_2d[0,1] #element with row index 0 and column index 1
```

Wrapping Up

In this activity, we learned about lists, `arrays`, dictionaries, sets, and tuples and were introduced to the concept of DataFrames. In future activities we will build on these concepts until you feel comfortable working with and manipulating large groups of data.